МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «МИРЭА — РОССИЙСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»
(РТУ МИРЭА)

ОТЧЁТ
О ПРАКТИЧЕСКОЙ РАБОТЕ
по теме:
РАБОТА С MONGODB: ПОДКЛЮЧЕНИЕ, СОЗДАНИЕ КОЛЛЕКЦИИ, CRUD-ОПЕРАЦИИ

Исполнитель НИР,
Студент группы ЭФМО-01-26 _____ Акулов Роман Витальевич
подпись, дата

Москва 2026

СОДЕРЖАНИЕ

1 Теоретические сведения 4

2 Подготовка программного окружения 6

2.1 Проверка Go 6

2.2 Проверка Docker Desktop 6

3 Создание проекта 8

3.1 Подготовка каталогов 8

3.2 Инициализация Go-модуля и зависимости 8

3.3 Настройка параметров приложения 9

4 Запуск MongoDB в Docker 11

5 Реализация приложения на Go 13

5.1 Подключение к MongoDB 13

5.2 Модель заметки 15

5.3 Репозиторий, коллекция и индекс 15

5.4 HTTP-обработчики и маршруты 17

5.5 Точка входа приложения 17

6 Запуск приложения 19

7 Проверка CRUD-операций 21

7.1 Create — создание заметки 21

7.2 Read — получение списка заметок 22

7.3 Read — получение заметки по идентификатору 23

7.4 Update — частичное обновление заметки 23

7.5 Delete — удаление заметки 24

8 Проверка уникального индекса и обработки ошибок 26

9 Дополнительное задание: aggregation pipeline 28

10 Автоматизированный тест 29

11 Остановка приложения и контейнера 30

12 Ответы на контрольные вопросы 31

1 Теоретические сведения

В этой работе я использую MongoDB — документно-ориентированную NoSQL-СУБД. Основное отличие от реляционного подхода заключается в способе хранения информации: вместо таблиц со строками и столбцами MongoDB использует документы, объединённые в коллекции.

Документ по своей структуре напоминает JSON-объект. При этом внутри MongoDB данные хранятся в формате BSON (Binary JSON), который, помимо обычных строк, чисел, массивов и вложенных объектов, поддерживает специальные типы. Один из них — `ObjectID`, используемый для идентификации документов. Коллекция представляет собой набор таких документов. В отличие от реляционной таблицы, документы одной коллекции не обязаны иметь полностью одинаковый набор полей.

Каждый документ содержит поле `_id`. Если я не указываю его при добавлении записи, MongoDB генерирует идентификатор самостоятельно. В Go для работы с ним используется `primitive.ObjectID`, а строковое шестнадцатеричное представление преобразуется функцией `primitive.ObjectIDFromHex`.

Для управления заметками применяются четыре группы операций CRUD:

- **Create** — добавление документов с помощью `InsertOne`;
- **Read** — чтение одного документа или набора документов через `FindOne` и `Find`;
- **Update** — изменение документа методами `UpdateOne` и `FindOneAndUpdate`;
- **Delete** — удаление документа с помощью `DeleteOne`.

Отдельно в работе используется индекс по полю `title`. Он ускоряет поиск по этому полю и, благодаря уникальному ограничению, не позволяет сохранить две заметки с одинаковым заголовком. Если приложение отправит повторяющийся заголовок, MongoDB отклонит вставку. На уровне HTTP API такая ситуация преобразуется в статус `409 Conflict`.

Для запросов к базе я использую `context.Context` с ограничением времени выполнения. Это важно при сетевых сбоях или недоступности

сервера: операция не должна бесконечно удерживать обработчик HTTP-запроса в ожидании ответа.

2 Подготовка программного окружения

2.1 Проверка Go

Для проекта требуется Go версии 1.21 или новее. После установки проверяю, что компилятор доступен из PowerShell:

```
go version
```

Листинг 1 — Команда проверки установленной версии Go

Команда должна вывести установленную версию. Если PowerShell не находит `go`, необходимо проверить установку и наличие каталога Go в системной переменной `PATH`.

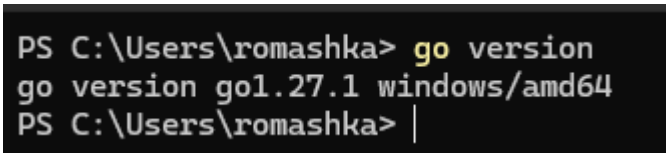


Рисунок 1 — Проверка версии Go в PowerShell

2.2 Проверка Docker Desktop

MongoDB запускаю в контейнере, поэтому заранее устанавливаю Docker Desktop и дожидаюсь запуска Docker Engine. Для проверки выполняю:

```
docker --version
docker compose version
```

Листинг 2 — Команды проверки Docker и Docker Compose

Обе команды должны вернуть номера версий. Если Docker Engine не запускается, проверяю настройки виртуализации в BIOS/UEFI и работоспособность WSL 2, когда Docker Desktop использует её в качестве backend.



Рисунок 2 — Проверка Docker и Docker Compose

3 Создание проекта

3.1 Подготовка каталогов

Рабочий каталог проекта создаю на рабочем столе. Внутри сразу разделяю точку входа приложения, подключение к базе и логику работы с заметками:

```
mkdir -p pz8-mongo
cd ./pz8-mongo
mkdir -p \\
    ./cmd/api \\
    ./internal/db \\
    ./internal/notes
```

Листинг 3 — Создание каталогов проекта

Каталог `cmd/api` предназначен для запуска HTTP-сервера. В `internal/db` размещается код подключения к MongoDB, а в `internal/notes` — модель заметки, репозиторий, HTTP-обработчики и тесты.



Рисунок 3 — Созданная структура каталогов проекта

3.2 Инициализация Go-модуля и зависимости

В корне проекта создаю Go-модуль:

```
go mod init example.com/pz8-mongo
```

Листинг 4 — Инициализация Go-модуля

После этого появляется `go.mod`. Он хранит имя модуля и сведения о зависимостях. Для проекта устанавливаю необходимые библиотеки:

```
go get github.com/go-chi/chi/v5
go get github.com/joho/godotenv
go get go.mongodb.org/mongo-driver@v1.17.6
```

Листинг 5 — Установка библиотек проекта

`chi` отвечает за маршрутизацию HTTP-запросов, `godotenv` загружает параметры из `.env`, а `mongo-driver` предоставляет API для работы с MongoDB.

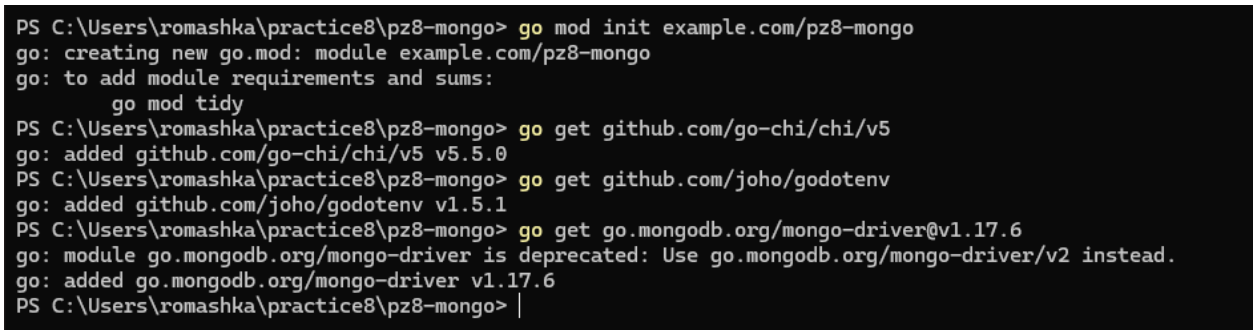


Рисунок 4 — Инициализация модуля и установка библиотек

3.3 Настройка параметров приложения

В корне создаю `.env.example`:

```
MONGO_URI=mongodb://root:secret@localhost:27017/?authSource=admin
MONGO_DB=pz8
HTTP_ADDR=:8080
```

Листинг 6 — Пример конфигурации приложения

Здесь задаются адрес MongoDB с параметрами авторизации, имя используемой базы и адрес HTTP-сервера. На основе примера создаю локальный файл настроек:

```
cp .env.example .env
```

Листинг 7 — Создание локального файла конфигурации

В рабочем `.env` хранятся локальные параметры подключения, поэтому его не следует добавлять в Git. Для этого файл `.env` указывается в `.gitignore`.

Сам `.env.example`, напротив, можно хранить в репозитории: он показывает необходимые переменные, но не заменяет локальную конфигурацию.

В URI параметр `authSource=admin` указывает, что пользователь `root` аутентифицируется через административную базу MongoDB.

```
romashka@WSL:~$ nano .env.example
romashka@WSL:~$ cp .env.example .env
romashka@WSL:~$ cat .env
MONGO_URI=mongodb://root:secret@localhost:27017/?authSource=admin
MONGO_DB=pz8
HTTP_ADDR=:8080
```

Рисунок 5 — Пример конфигурации и локальный файл `.env`

4 Запуск MongoDB в Docker

В корне проекта создаю `docker-compose.yml`, описывающий контейнер базы данных:

```yaml
services:
  mongo:
    image: mongo:7
    container_name: mongo-dev
    restart: unless-stopped
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: root
      MONGO_INITDB_ROOT_PASSWORD: secret
    volumes:
      - mongo_data:/data/db

volumes:
  mongo_data:
```

Листинг 8 — Конфигурация контейнера MongoDB в Docker Compose

Для запуска используется образ MongoDB версии 7. Порт `27017` внутри контейнера доступен на таком же порту компьютера. Переменные окружения задают учётные данные администратора, а именованный том `mongo_data` сохраняет содержимое базы между перезапусками контейнера.

Запускаю сервис в фоновом режиме:

```
docker compose up -d
```

Листинг 9 — Запуск MongoDB в фоновом режиме

Проверяю, что контейнер создан и работает:

```
docker compose ps
```

Листинг 10 — Проверка состояния контейнера

В выводе должен присутствовать `mongo-dev` со статусом `Up` или `running`, а порт `27017` должен быть опубликован.

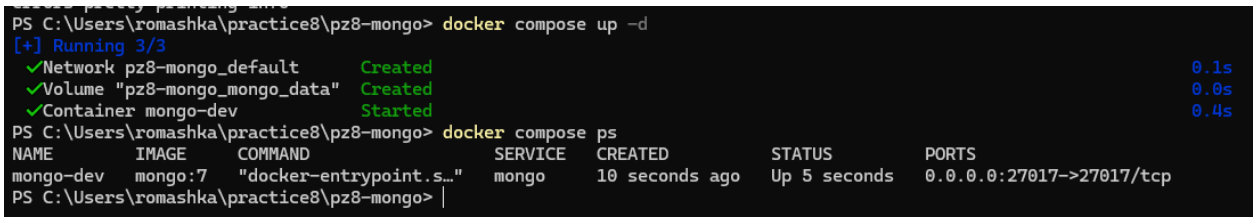


Рисунок 6 — Запуск MongoDB и проверка контейнера

Дополнительно проверяю доступность сервера через консоль MongoDB:

```
docker exec -it mongo-dev mongosh -u root -p secret --
authenticationDatabase admin
```

Листинг 11 — Подключение к MongoDB через mongosh

После подключения выполняю:

```
show dbs
exit
```

Листинг 12 — Проверка списка баз данных и выход из оболочки

Если оболочка открывается и команда `show dbs` выполняется, сервер доступен. При этом базы `pz8` в списке может ещё не быть: MongoDB фактически создаёт базу после появления в ней первой записи.

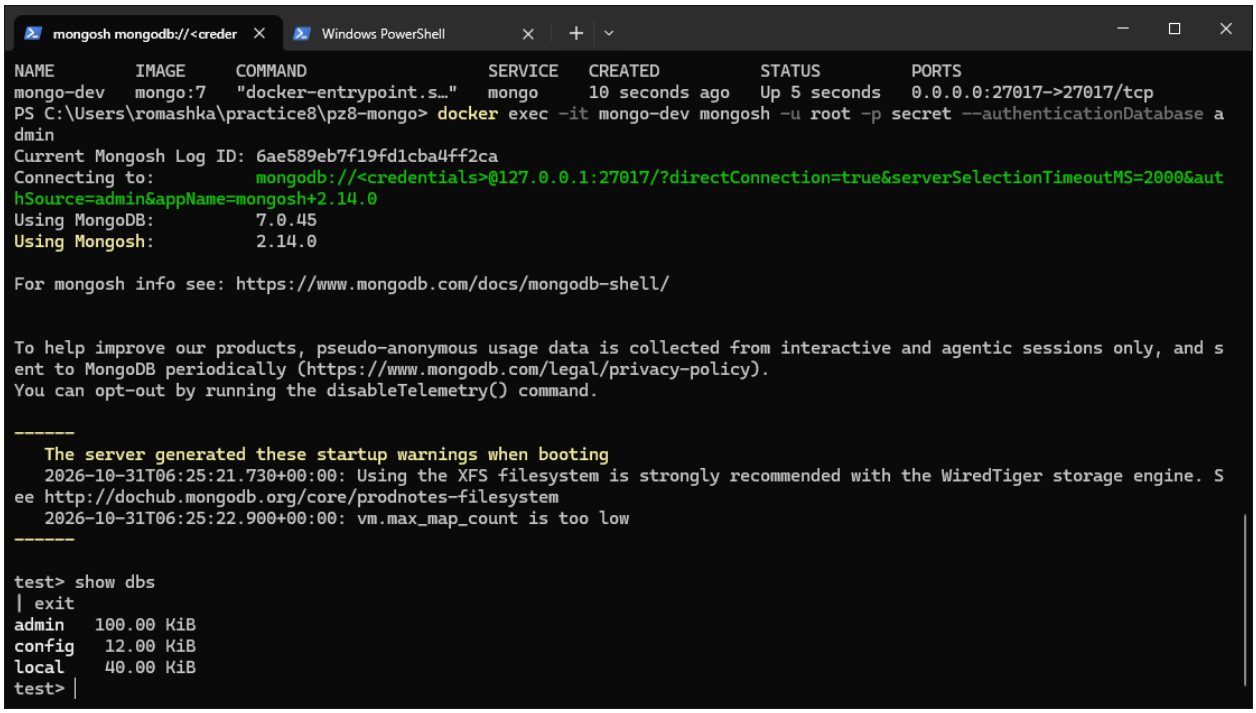


Рисунок 7 — Подключение к MongoDB через `mongosh`

5 Реализация приложения на Go

5.1 Подключение к MongoDB

Для подключения создаю файл `internal/db/mongo.go`. Функция `ConnectMongo` принимает родительский контекст, URI и имя базы. Сначала она устанавливает таймаут на подключение, затем отдельно проверяет доступность сервера методом `Ping`.

```go
package db

import (
    "context"
    "time"

    "go.mongodb.org/mongo-driver/mongo"
    "go.mongodb.org/mongo-driver/mongo/options"
)

type MongoDeps struct {
    Client   *mongo.Client
    Database *mongo.Database
}

func ConnectMongo(ctx context.Context, uri, dbName string)
(*MongoDeps, error) {
    // Ограничиваю время установления соединения.
    dialCtx, cancel := context.WithTimeout(ctx, 10*time.Second)
    defer cancel()

    client, err := mongo.Connect(
        dialCtx,
        options.Client().ApplyURI(uri),
    )
    if err != nil {
        return nil, err
    }

    // Проверяю, что сервер действительно отвечает.
    pingCtx, cancelPing := context.WithTimeout(ctx, 3*time.Second)
    defer cancelPing()

    if err := client.Ping(pingCtx, nil); err != nil {
        _ = client.Disconnect(context.Background())
        return nil, err
    }

    return &MongoDeps{
        Client:   client,
        Database: client.Database(dbName),
    }, nil
}
```

Листинг 13 — Реализация подключения к MongoDB в файле `mongo.go`

Структура `MongoDeps` хранит клиент и объект выбранной базы данных. Если `Ping` завершается ошибкой, соединение закрывается, а ошибка передаётся вызывающему коду. Благодаря этому приложение может обнаружить недоступность MongoDB уже при запуске, а не при обработке первого запроса пользователя.

5.2 Модель заметки

Структуру документа описываю в `internal/notes/model.go`:

```
package notes

import (
    "time"

    "go.mongodb.org/mongo-driver/bson/primitive"
)

type Note struct {
    ID        primitive.ObjectID `bson:"_id,omitempty" json:"id"`
    Title     string             `bson:"title" json:"title"`
    Content   string             `bson:"content" json:"content"`
    CreatedAt time.Time          `bson:"createdAt" json:"createdAt"`
    UpdatedAt time.Time          `bson:"updatedAt" json:"updatedAt"`
}
```

Листинг 14 — Модель заметки в файле `model.go`

Теги `bson` определяют имена полей при сохранении документа в MongoDB, а `json` — имена свойств в HTTP-ответах. Для идентификатора используется `primitive.ObjectID`. Тег `omitempty` позволяет не передавать `_id` при первой вставке: MongoDB самостоятельно назначит значение.

В модели также предусмотрена структура `Stats`, которая используется дополнительным методом получения статистики.

5.3 Репозиторий, коллекция и индекс

Доступ к коллекции `notes` вынесен в `internal/notes/repo.go`. Репозиторий получает коллекцию через `database.Collection("notes")`, после чего создаёт уникальный индекс по `title`:

```
        _, err := collection.Indexes().CreateOne(
            context.Background(),
            mongo.IndexModel{
                Keys: bson.D{
                    {Key: "title", Value: 1},
                },
                Options: options.Index().
                    SetUnique(true).
                    SetName("uniq_note_title"),
            },
        )
```

Листинг 15 — Создание уникального индекса по полю `title`

Значение `1` означает сортировку ключей индекса по возрастанию. Вызов `SetUnique(true)` включает ограничение уникальности. Если в коллекции уже есть заметки с одинаковым заголовком, создать такой индекс не получится, пока дубликаты не будут устранены.

В репозитории собраны операции, связанные с заметками:

- `Create` формирует документ, заполняет время создания и изменения, выполняет `InsertOne` и возвращает присвоенный идентификатор;
- `ByID` преобразует строковый идентификатор в `ObjectID` и ищет документ по `_id`;
- `List` возвращает список заметок, поддерживает поиск по заголовку без учёта регистра, сортировку и пагинацию;
- `Update` изменяет только переданные поля с помощью `$set` и обновляет `updatedAt`;
- `Delete` удаляет документ по `ObjectID` и проверяет количество удалённых записей;
- `GetStats` запускает aggregation pipeline и рассчитывает количество документов и среднюю длину их содержимого.

Для изменения документа используется `FindOneAndUpdate` с `SetReturnDocument(options.After)`. Поэтому метод возвращает уже обновлённую запись, а не её предыдущее состояние. Если документ не найден, репозиторий возвращает собственную ошибку `ErrNotFound`.

5.4 HTTP-обработчики и маршруты

В `internal/notes/handler.go` находятся обработчики HTTP-запросов. Для регистрации маршрутов применяется библиотека `chi`.

Метод	Маршрут	Назначение	Успешный статус
`POST`	`/api/v1/notes`	Создать заметку	`201 Created`
`GET`	`/api/v1/notes`	Получить список, выполнить поиск и пагинацию	`200 OK`
`GET`	`/api/v1/notes/{id}`	Получить заметку по идентификатору	`200 OK`
`PATCH`	`/api/v1/notes/{id}`	Частично изменить заметку	`200 OK`
`DELETE`	`/api/v1/notes/{id}`	Удалить заметку	`204 No Content`
`GET`	`/api/v1/notes/stats`	Получить статистику	`200 OK`

При обращении к репозиторию обработчик создаёт контекст с таймаутом 5 секунд. Это ограничивает время ожидания ответа от MongoDB.

При создании заметки проверяется корректность JSON и наличие заголовка. Если JSON некорректен, API возвращает `400 Bad Request`. Если заголовок уже занят, возвращается `409 Conflict`. Для отсутствующей заметки и некорректного `ObjectID` используется `404 Not Found`.

Для `PATCH` поля описаны через указатели `*string`. Это позволяет отличить отсутствующее свойство JSON от свойства, которому явно передали пустую строку. В результате изменяются только те поля, которые действительно присутствуют в запросе.

5.5 Точка входа приложения

Запуск сервиса находится в `cmd/api/main.go`. В функции `main` последовательно выполняются основные действия: загружаются параметры из `.env`, считываются URI MongoDB, имя базы и адрес HTTP-сервера, устанавливается соединение и проверяется его доступность. Затем создаётся

репозиторий с индексом, регистрируются маршруты проверки состояния и управления заметками, после чего запускается HTTP-сервер на порту `8080`.

При завершении приложения предусмотрено закрытие HTTP-сервера и соединения с MongoDB. Для отдельной проверки HTTP-сервера регистрируется маршрут `/health`, возвращающий JSON:

```
{"status":"ok"}
```

Листинг 16 — Пример ответа маршрута `/health`

6 Запуск приложения

Перед запуском Go-сервера убеждаюсь, что MongoDB продолжает работать:

```
docker compose ps
```

Листинг 17 — Проверка состояния MongoDB перед запуском приложения

Затем из корня проекта загружаю и проверяю зависимости, после чего запускаю приложение:

```
go mod tidy
go run ./cmd/api
```

Листинг 18 — Подготовка зависимостей и запуск HTTP-сервера

`go mod tidy` приводит зависимости и импорты Go-модуля в согласованное состояние. Команда `go run ./cmd/api` собирает и запускает HTTP-сервис. При успешном старте в терминале появляется сообщение о прослушивании адреса `:8080`.

Окно с работающим сервером оставляю открытым. Проверочные запросы выполняю во втором окне PowerShell:

```
Invoke-RestMethod -Uri 'http://localhost:8080/health'
```

Листинг 19 — Проверка доступности HTTP-сервера

Ожидаемый ответ:

```
status
------
ok
```

Листинг 20 — Ожидаемый ответ маршрута `/health`

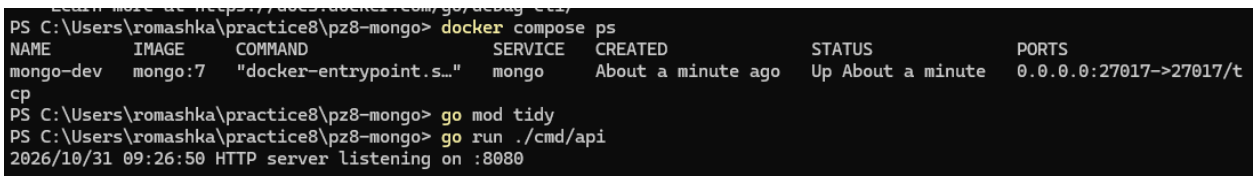


Рисунок 8 — Запуск HTTP-сервера



Рисунок 9 — Проверка маршрута `/health`

7 Проверка CRUD-операций

Для проверки API использую PowerShell. Результат создания заметки сохраню в `$created`, а её идентификатор — в `$id`. Все команды этого раздела выполняю в одном окне PowerShell, чтобы переменные оставались доступными.

7.1 Create — создание заметки

Отправляю запрос `POST` с заголовком и содержимым новой заметки:

```
$body = @{
    title   = 'First note'
    content = 'Hello Mongo!'
} | ConvertTo-Json

$created = Invoke-RestMethod `
    -Method Post `
    -Uri 'http://localhost:8080/api/v1/notes' `
    -ContentType 'application/json' `
    -Body $body

$created
$id = $created.id
```

Листинг 21 — Создание заметки и сохранение её идентификатора

В ответе должны присутствовать `id`, `title`, `content`, `createdAt` и `updatedAt`. Идентификатор назначает MongoDB, а успешное создание обозначается статусом `201 Created`.

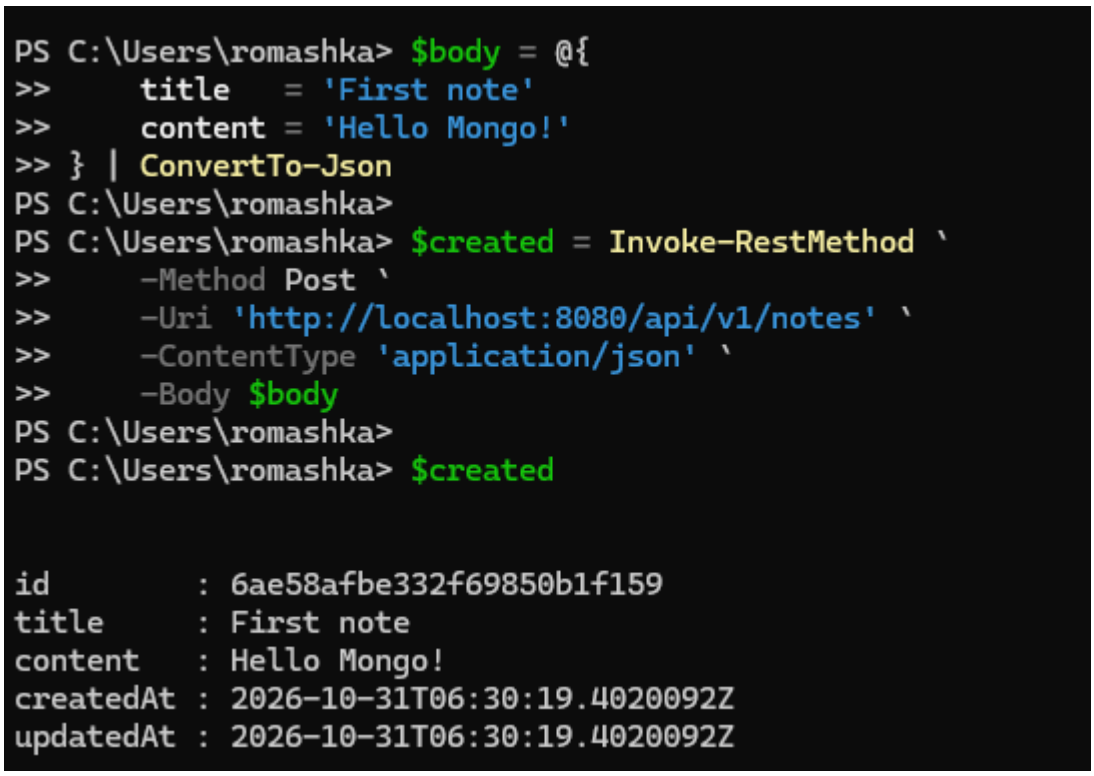


Рисунок 10 — Создание заметки и получение её ObjectID

7.2 Read — получение списка заметок

Сначала запрашиваю до пяти записей, начиная с первой:

```
Invoke-RestMethod -Uri 'http://localhost:8080/api/v1/notes?limit=5&
skip=0'
```

Листинг 22 — Получение списка заметок с ограничением количества

Параметр `limit` ограничивает количество возвращаемых документов, а `skip` задаёт число записей, которые нужно пропустить. По умолчанию API возвращает не более 20 документов, верхняя граница `limit` составляет 200.

Проверяю фильтрацию по заголовку:

```
Invoke-RestMethod -Uri 'http://localhost:8080/api/v1/notes?limit=5&
skip=0&q=first'
```

Листинг 23 — Поиск заметок по заголовку

Параметр `q` передаёт строку поиска по полю `title`. Сопоставление выполняется без учёта регистра, поэтому запрос с `first` находит заголовок `First note`.



Рисунок 11 — Получение списка с пагинацией и поиском

7.3 Read — получение заметки по идентификатору

Использую идентификатор, который сохранил после создания:

```
Invoke-RestMethod -Uri "http://localhost:8080/api/v1/notes/$id"
```

Листинг 24 — Получение заметки по ObjectID

Если документ существует, сервис возвращает его поля. При некорректном идентификаторе или отсутствии документа API отвечает статусом `404 Not Found`.

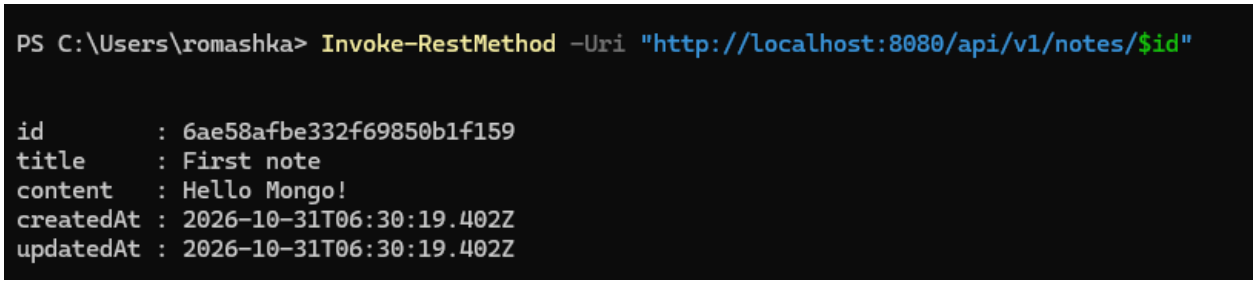


Рисунок 12 — Чтение заметки по ObjectID

7.4 Update — частичное обновление заметки

Теперь меняю только содержимое. Поле `title` в запрос не включаю:

```
$patch = @{
    content = 'Updated content'
} | ConvertTo-Json

$updated = Invoke-RestMethod `
    -Method Patch `
    -Uri "http://localhost:8080/api/v1/notes/$id" `
    -ContentType 'application/json' `
    -Body $patch

$updated
```

Листинг 25 — Частичное обновление содержимого заметки

Обработчик передаёт в MongoDB обновление с оператором `$set`. Поскольку в теле запроса есть только `content`, заголовок остаётся прежним. Значение `updatedAt` при этом изменяется.

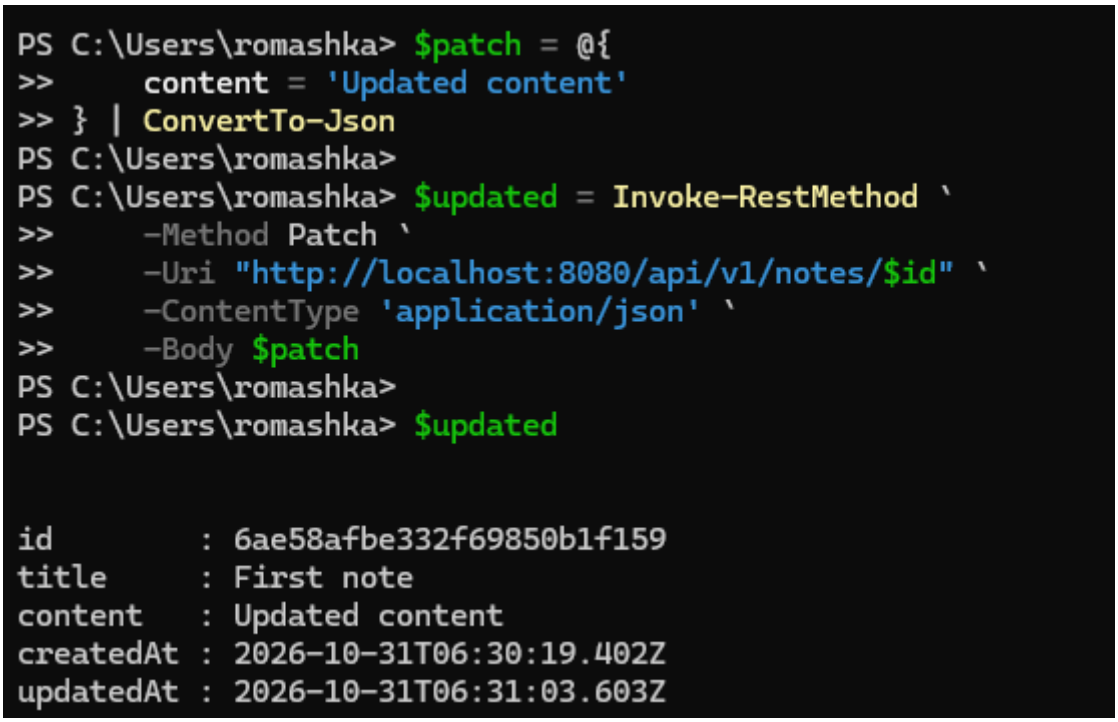


Рисунок 13 — Частичное обновление содержимого заметки

7.5 Delete — удаление заметки

Удаляю созданную запись:

```
Invoke-WebRequest `
    -Method Delete `
    -Uri "http://localhost:8080/api/v1/notes/$id"
```

Листинг 26 — Удаление заметки

При успешном выполнении сервер возвращает `204 No Content`. Такой ответ означает, что документ удалён, а тело ответа не требуется.

Чтобы убедиться, что запись действительно больше не доступна, повторяю запрос чтения:

```
try {
    Invoke-RestMethod -Uri "http://localhost:8080/api/v1/notes/$id"
}
catch {
    $_.Exception.Response.StatusCode.value__
}
```

Листинг 27 — Проверка отсутствия удалённой заметки

Ожидаемый результат — `404`: документ с таким идентификатором уже отсутствует в коллекции.

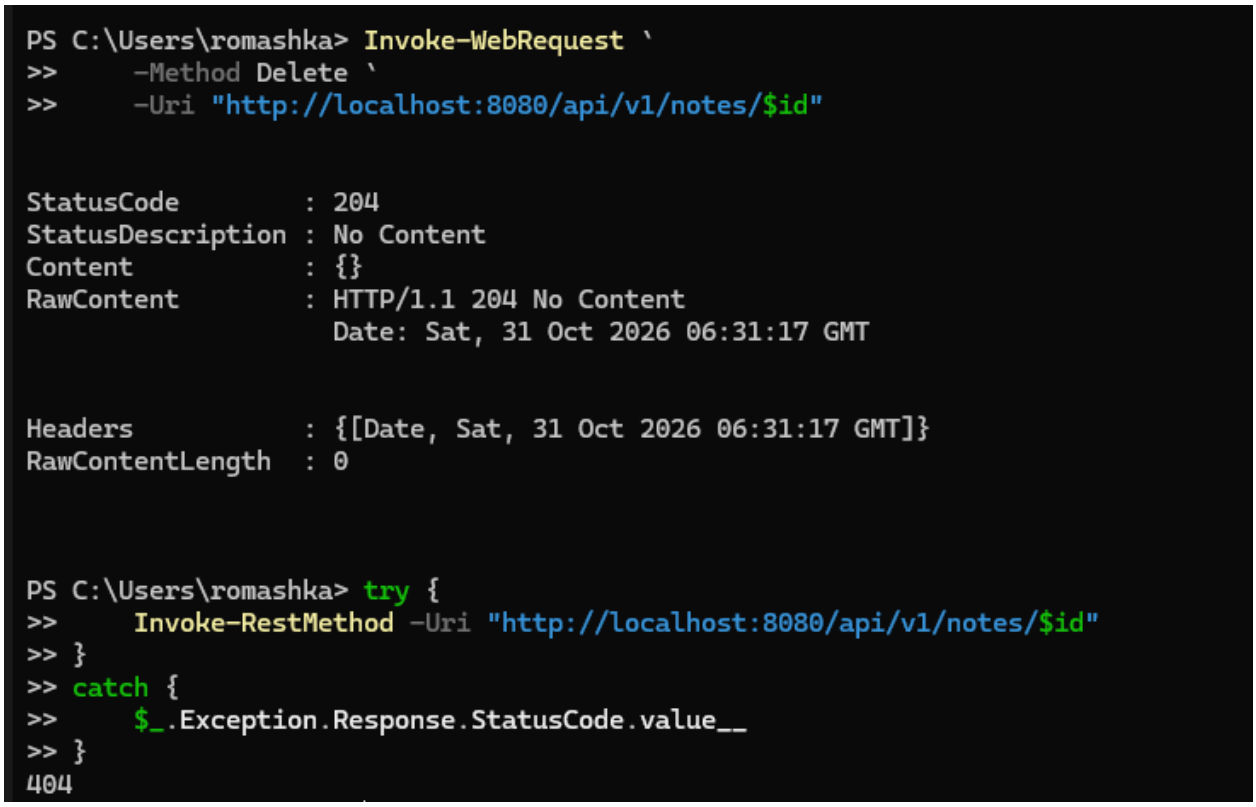


Рисунок 14 — Удаление заметки и проверка результата

8 Проверка уникального индекса и обработки ошибок

Проверяю ограничение уникальности на практике. Сначала отправляю запрос на создание заметки с заголовком `Unique title test`:

```
$duplicateBody = @{
    title   = 'Unique title test'
    content = 'First'
} | ConvertTo-Json

Invoke-RestMethod `
    -Method Post `
    -Uri 'http://localhost:8080/api/v1/notes' `
    -ContentType 'application/json' `
    -Body $duplicateBody
```

Листинг 28 — Создание первой заметки с уникальным заголовком

Затем пытаюсь добавить ещё одну заметку с тем же заголовком:

```
$duplicateBody2 = @{
    title   = 'Unique title test'
    content = 'Second'
} | ConvertTo-Json

try {
    Invoke-RestMethod `
        -Method Post `
        -Uri 'http://localhost:8080/api/v1/notes' `
        -ContentType 'application/json' `
        -Body $duplicateBody2
}
catch {
    $_.Exception.Response.StatusCode.value__
}
```

Листинг 29 — Попытка повторного создания заметки с тем же заголовком

Вторая вставка должна завершиться ответом `409`. Это подтверждает, что уникальный индекс действительно запрещает повторяющиеся заголовки, а HTTP-обработчик корректно преобразует ошибку MongoDB в статус `Conflict`.

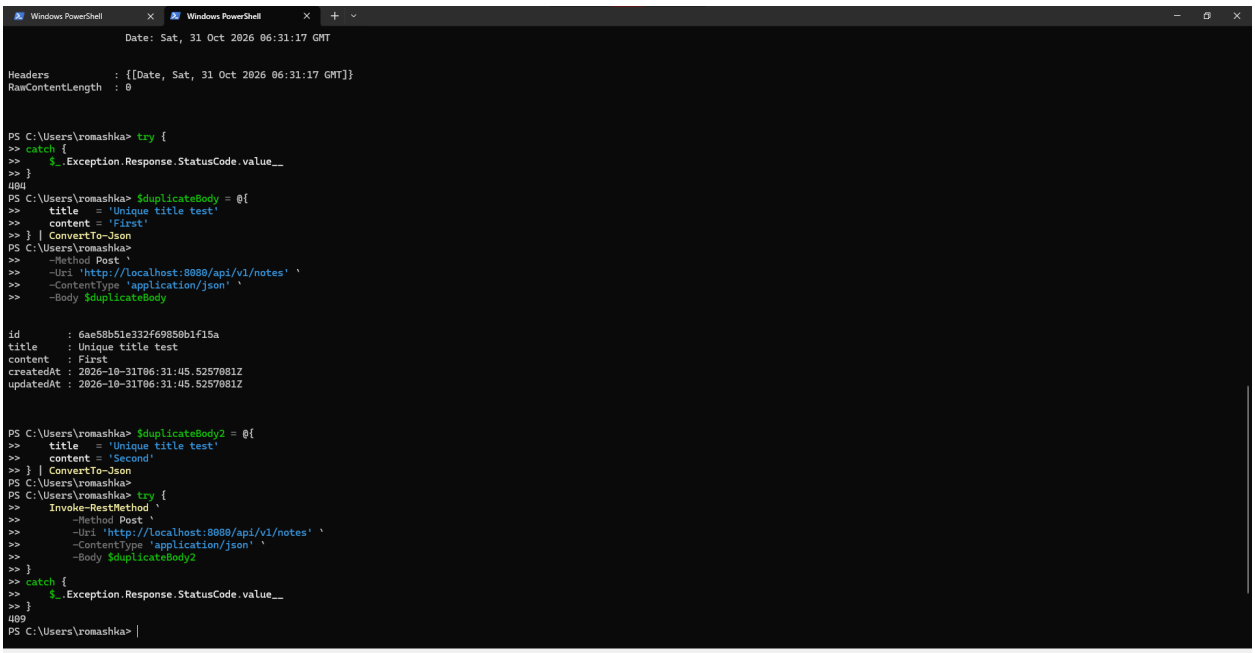


Рисунок 15 — Проверка уникальности заголовка

9 Дополнительное задание: aggregation pipeline

В приложении предусмотрен отдельный маршрут статистики. Вызываю его следующим запросом:

```
Invoke-RestMethod -Uri 'http://localhost:8080/api/v1/notes/stats'
```

Листинг 30 — Получение статистики заметок

В репозитории для расчёта используется aggregation pipeline MongoDB. На этапе `$group` вычисляются два значения: `count` — количество документов, и `averageContentLength` — средняя длина содержимого. Для среднего значения применяются `$avg` и `$strLenCP`.

Пример структуры ответа:

```
{
    "count": 2,
    "averageContentLength": 18.5
}
```

Листинг 31 — Пример структуры ответа статистики

Числа здесь приведены только для демонстрации формата. Фактический результат зависит от содержимого коллекции на момент запроса. Если документов нет, endpoint возвращает нулевые значения.



Рисунок 16 — Результат агрегации статистики заметок

10 Автоматизированный тест

Для проверки операций репозитория добавлен интеграционный тест `internal/notes/repo_test.go`. Он проходит полный сценарий работы с документом: создание заметки, чтение по `ObjectID`, частичное обновление и удаление.

Тестовая база получает отдельное имя, в которое включаются текущие дата и время. После выполнения теста она удаляется. За счёт этого проверка не затрагивает основную базу `pz8`.

В PowerShell задаю URI MongoDB для текущего сеанса и запускаю тесты:

```
# (env): MONGO_URI = 'mongodb://root:secret@localhost:27017/?
authSource=admin'
go test ./...
```

Листинг 32 — Запуск интеграционных тестов репозитория

Если `MONGO_URI` не задана, интеграционный тест пропускается, поэтому обычная проверка пакетов не требует запущенного сервера MongoDB. При работающем контейнере и корректном подключении тест должен завершиться успешно.

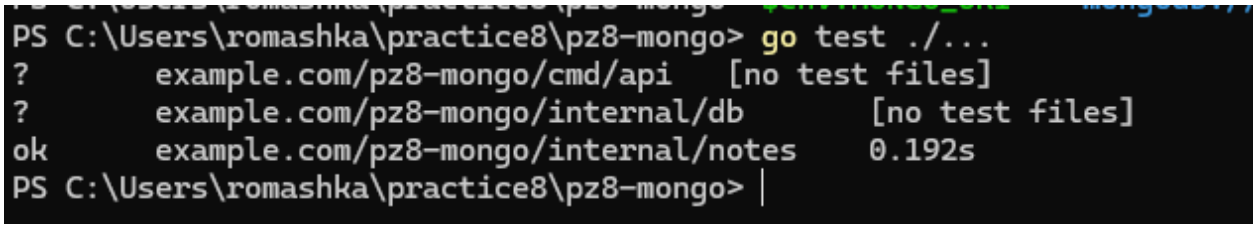


Рисунок 17 — Результат выполнения `go test ./...`

11 Остановка приложения и контейнера

Сначала останавливаю HTTP-сервер сочетанием `Ctrl+C` в терминале, где выполнялась команда `go run`. Затем из каталога проекта останавливаю MongoDB:

```
docker compose down
```

Листинг 33 — Остановка контейнера MongoDB

Эта команда удаляет контейнер, но оставляет именованный том `mongo_data`, поэтому данные сохраняются и будут доступны при следующем запуске.

Если требуется удалить вместе с контейнером и сохранённые данные, можно выполнить:

```
docker compose down -v
```

Листинг 34 — Остановка MongoDB с удалением томов

В обычном завершении практической работы использую первый вариант, чтобы не удалять содержимое базы без необходимости.

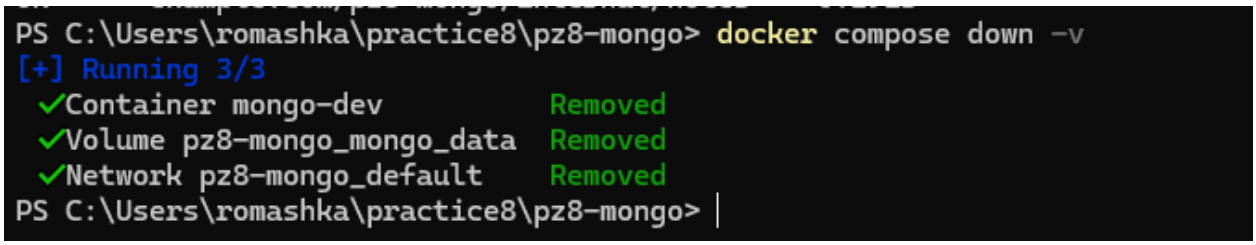


Рисунок 18 — Остановка контейнера MongoDB

12 Ответы на контрольные вопросы

1. Чем документная модель MongoDB принципиально отличается от реляционной? Когда она удобнее?

В реляционной базе информация организована в таблицы с фиксированными столбцами и связями между ними. MongoDB хранит записи в виде BSON-документов, объединённых в коллекции. Документы могут содержать вложенные объекты и массивы, а их набор полей не обязан полностью совпадать.

Такой подход удобен, когда данные естественным образом имеют вложенную структуру или схема может меняться. Например, это могут быть профили пользователей, заметки или карточки товаров. Реляционная модель чаще подходит для сценариев, где особенно важны сложные связи между сущностями, строгая целостность и транзакционная обработка.

2. Что такое ObjectID и зачем нужен `_id`? Как корректно парсить и валидировать его в Go?

Поле `_id` служит идентификатором документа. Если его не передать при вставке, MongoDB сформирует значение автоматически. `ObjectID` — специальный тип идентификатора, в котором содержатся временная составляющая, случайное значение и счётчик.

В Go для него используется `primitive.ObjectID`. Строковое значение преобразуется вызовом `primitive.ObjectIDFromHex(idHex)`. Если функция возвращает ошибку, строка не соответствует формату ObjectID. В рассматриваемом API некорректный идентификатор обрабатывается как `404 Not Found`.

3. Какие операции CRUD предоставляет драйвер MongoDB и какие операторы обновления известны?

Для добавления документов применяются `InsertOne` и `InsertMany`, для чтения — `FindOne` и `Find`, для обновления — `UpdateOne`, `UpdateMany` и `FindOneAndUpdate`, для удаления — `DeleteOne` и `DeleteMany`.

При обновлении документов можно использовать разные операторы. `$set` задаёт значения полей, `$inc` увеличивает числовое значение, `$push` добавляет элемент в массив, а `$pull` удаляет из массива элементы,

соответствующие условию. В этой работе для частичного изменения заметки используется `$set`.

4. Как устроены индексы в MongoDB? Как создать уникальный индекс и чем он грозит при вставке?

Индекс представляет собой дополнительную структуру данных, которая помогает быстрее находить документы по выбранным полям. MongoDB поддерживает одиночные, составные, уникальные и TTL-индексы.

Для создания индекса используется `collection.Indexes().CreateOne(...)`, которому передаётся `mongo.IndexModel`. Уникальность включается параметром `SetUnique(true)`. В проекте индекс построен по полю `title`. Если вставить документ с уже существующим заголовком, MongoDB отклонит операцию из-за дублирования ключа. Приложение преобразует эту ошибку в HTTP `409 Conflict`.

5. Почему важно использовать `context.WithTimeout` при вызовах к базе? Что произойдёт при его срабатывании?

`context.WithTimeout` задаёт максимальное время, в течение которого операция может выполняться. Это защищает приложение от бесконечного ожидания при сетевых проблемах или недоступности MongoDB.

Когда срок действия контекста истекает, он отменяется, а операция завершается ошибкой отмены или превышения времени ожидания. Приложение получает управление обратно и может освободить связанные ресурсы либо вернуть ошибку вызывающему коду.